

WillTalk

Plataforma de Atendimento e Triagem via WhatsApp

Documentacao Tecnica Completa

Arquitetura, fluxos, integracoes e estado atual do sistema

Projeto	WillTalk - Suporte Pro
Repositorio	C:\willydev\willtalk
Versao do pacote	0.1.0
Stack principal	Next.js 16 - React 19 - PostgreSQL - Socket.IO
Ambiente atual	Desenvolvimento / homologacao local (porta 4001 + 4002)
Data do levantamento	22 de julho de 2026
Base do levantamento	Leitura integral do codigo-fonte em disco

Sumario

1. Sumario executivo	5
1.1 Numeros do sistema	5
1.2 Maturidade por area	5
2. Arquitetura do sistema	6
2.1 Visao macro	6
2.2 Processos em execucao	6
2.3 Camadas do codigo	7
3. Stack tecnologica	8
3.1 Backend	8
3.2 Frontend (SPA ativa)	8
4. Estrutura do repositorio	9
4.1 Modulos de lib/ por responsabilidade	9
5. Modelo de dados	11
5.1 Entidades e relacionamentos	11
5.2 Tabela: conversations	11
5.3 Tabela: tickets	11
5.4 Indices relevantes	12
5.5 Camada de acesso a dados	12
6. Autenticacao, sessao e permissoes	13
6.1 Fluxo de login	13
6.2 Guardas do servidor	13
6.3 Matriz de papeis	13
6.4 Autenticacao maquina-a-maquina	13
7. Canais de WhatsApp	15
7.1 Modo unofficial (whatsapp-web.js + QR Code)	15
7.2 Modo twilio (oficial)	15
7.3 Estrategia de envio com fallback	16
8. Motor de triagem	17
8.1 Selecao do motor	17
8.2 Motor A — Orquestrador do Cerebro Operacional	17
8.3 Motor B — IA local com fallback em camadas	18
8.4 Motor C — menu numerico legado	18
8.5 Guardas anti-loop e deduplicacao	18

9. Fluxos de ponta a ponta	20
9.1 Fluxo 1 — Mensagem nova de cliente ate a resposta do bot	20
9.2 Fluxo 2 — Atendente responde pelo painel	20
9.3 Fluxo 3 — Encerramento e pesquisa de satisfacao	20
9.4 Fluxo 4 — Resposta assistida vinda do Cerebro	20
10. Referencia da API	22
10.1 Autenticacao e sessao	22
10.2 Conversas e mensagens	22
10.3 Cadastros e configuracao	22
10.4 Operacao e integracoes	23
10.5 Contrato do ticket-upsert	23
11. Integracoes externas	25
11.1 Webhook de saida (WillTalk para n8n / Cerebro)	25
11.2 Configuracao do n8n	25
11.3 Cloudinary	26
11.4 Gemini na interface do atendente	26
12. Tempo real	27
12.1 Estrategia de atualizacao da interface	27
13. Interfaces de usuario	28
13.1 SPA OmniDesk (interface ativa)	28
13.2 Interface Next legada	28
14. Regras de negocio consolidadas	29
14.1 Horario comercial	29
14.2 Normalizacao de telefone	29
14.3 Ciclo de vida do status da conversa	29
14.4 SLA	29
14.5 Variaveis de resposta rapida	29
15. Configuracao e operacao	31
15.1 Variaveis de ambiente	31
15.2 Procedimento de subida	32
15.3 Conectar o WhatsApp	33
15.4 Scripts de apoio	33
16. Avaliacao tecnica e recomendacoes	34
16.1 Pontos fortes	34
16.2 Divida tecnica	34

16.3 Riscos de seguranca a tratar	34
16.4 Escalabilidade	35
16.5 Governanca do repositório	35
16.6 Roteiro sugerido	35
17. Glossario	37

1. Sumario executivo

O **WillTalk** é uma plataforma multi-tenant de atendimento ao cliente por WhatsApp, construída para operações de suporte técnico. Ela recebe mensagens de clientes, aplica uma camada de triagem automatizada (bot / IA), transforma cada conversa em um ticket com fila, SLA e responsável, e entrega aos atendentes um inbox em tempo real para conduzir e encerrar o atendimento.

O diferencial arquitetural do sistema, no estado atual, é que a **inteligência de triagem foi externalizada**. O WillTalk deixou de ser um bot de menu numérico fixo e passou a ser um *orquestrador de canal + sistema de registro*: ele persiste tudo (contato, conversa, ticket, mensagem, auditoria), fala com o WhatsApp, e delega a decisão de *o que responder* a um serviço externo chamado **Cerebro Operacional** (projeto Mavo AI / chat-inteligente). Quando o Cerebro não está disponível, o WillTalk degrada graciosamente para uma triagem local assistida por IA de visão e, em último caso, para o menu numérico legado.

1.1 Numeros do sistema

Dimensao	Valor atual
Rotas de API (Next.js App Router)	26 route handlers
Modulos de biblioteca (lib/)	27 arquivos TypeScript
Modelos de dominio	11 (Organization, User, Channel, Contact, Queue, Conversation, Message, Ticket, QuickReply, BusinessHour, AuditLog)
Interfaces de usuario	2 (Next.js legado + SPA Vite 'OmniDesk Pro' — esta é a ativa)
Provedores de WhatsApp	2 (whatsapp-web.js via QR Code e Twilio via webhook)
Backends de persistencia	2 implementacoes paralelas (Firestore e PostgreSQL/Supabase)
Modos de triagem	3 (Orquestrador Cerebro, IA local com fallback, menu numerico legado)
Eventos de webhook de saida	4 (ticket_created, ticket_updated, message_received, message_sent)
Portas de rede	4002 (API + Socket.IO), 4001 (SPA em dev)

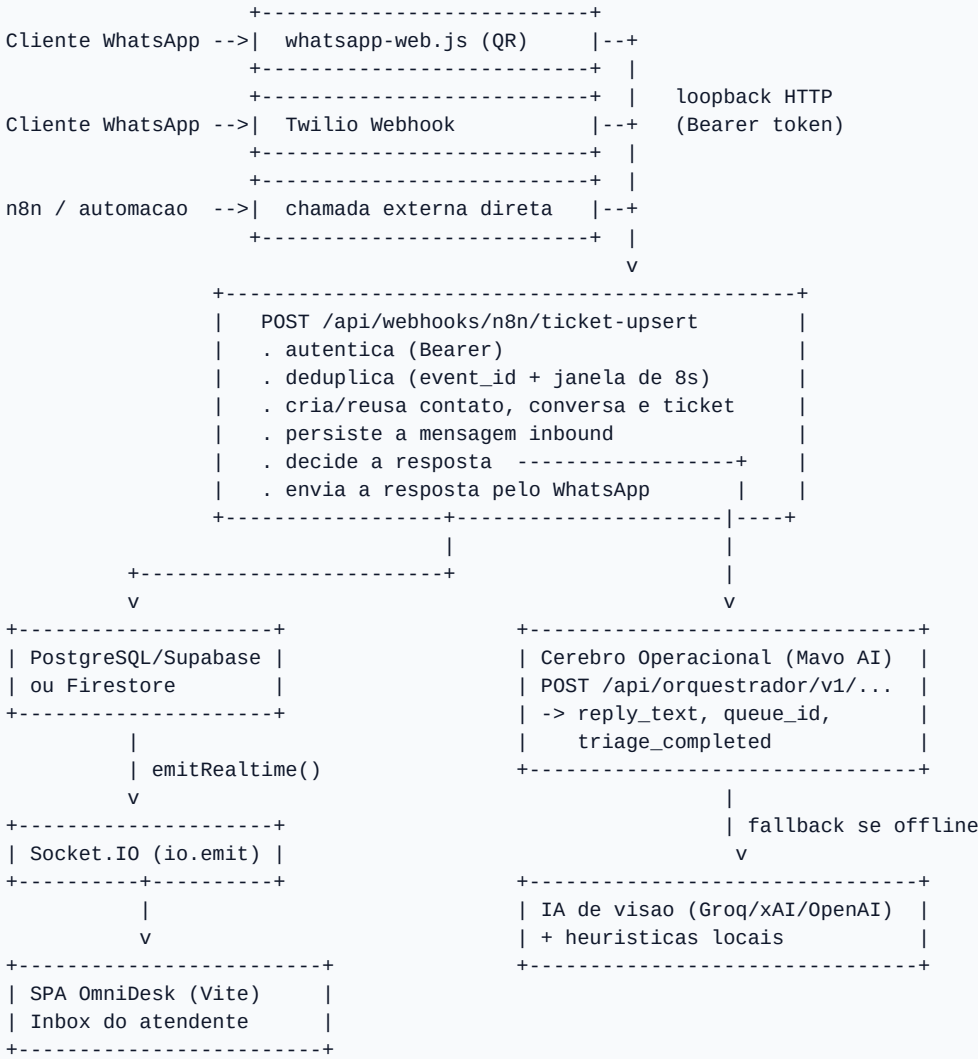
1.2 Maturidade por area

Area	Estado	Observacao
Ingestao WhatsApp (QR)	Producao	whatsapp-web.js com sessao persistida em .wwbj_auth, rate limit e filtros anti-ruído
Ingestao Twilio	Pronto	Webhook com validacao de assinatura; hoje inativo (provider = unofficial)
Triagem via Cerebro	Integrado	Cliente HTTP implementado e cabeado; depende do Cerebro estar no ar
Persistencia PostgreSQL	Producao	Schema SQL aplicado; camada supabase-repo com ~36 funcoes
Inbox do atendente (SPA)	Funcional	InboxConversations com ~51 KB de logica; tempo real via Socket.IO
Metricas / Dashboard	Parcial	API real de metricas existe; telas de Metricas e Cofre ainda usam mockData
SLA	Incompleto	Prazo (firstResponseDueAt) é gravado, mas nenhum job dispara o estouro
Workers BullMQ	Esqueleto	Somente a limpeza do Cloudinary faz trabalho real; os demais apenas logam
Testes automatizados	Ausente	Não há suite de testes; validacao é feita por scripts .bat manuais

2. Arquitetura do sistema

2.1 Visao macro

O sistema roda como um unico processo Node que embute o Next.js e o servidor Socket.IO (server.cjs), acompanhado opcionalmente de um processo de workers (worker.mjs) e de uma SPA servida pelo Vite em desenvolvimento. Todos os canais de entrada convergem para um unico ponto de decisao: a rota POST /api/webhooks/n8n/ticket-upsert.



2.2 Processos em execucao

Processo	Comando	Porta	Responsabilidade
Servidor principal	npm run dev	4002	Next.js (paginas + API) e servidor Socket.IO no mesmo HTTP server; hospeda tambem o cliente whatsapp-web.js em memoria global
SPA do atendente	cd frontend && npm run dev	4001	Vite + React Router; faz proxy de /api e /socket.io para 4002
Workers	npm run worker	-	BullMQ sobre Redis: limpeza de midia no Cloudinary, filas de SLA e webhook (hoje sem logica)

Detalhe importante de arquitetura: o cliente whatsapp-web.js vive dentro do processo Next, guardado em variáveis globais (global.__waClient, global.__waState). Isso mantém a sessão viva entre hot reloads em desenvolvimento, mas significa que **o sistema não é horizontalmente escalável hoje**: duas instâncias do servidor teriam duas sessões de WhatsApp e dois conjuntos de guardas anti-loop em memória.

2.3 Camadas do código

Camada	Onde vive	O que faz
Apresentacao (SPA)	frontend/	React 19 + React Router + Tailwind. Inbox, contatos, admin, painel de QR
Apresentacao (legado)	app/, components/	Paginas Next.js: /login, /dashboard, /dashboard/queues
API	app/api/**/route.ts	26 route handlers do App Router, todos com sessão via cookie JWT
Servicos	lib/services/	Fachadas finas: TicketService, ContactService, WhatsAppService, CloudinaryService, QuickReplyService
Dominio / regras	lib/*.ts	Triagem, horario comercial, webhooks, visao computacional, respostas rapidas
Persistencia	lib/repo.ts	Roteador que escolhe firestore-repo ou supabase-repo conforme DB_PROVIDER
Integracoes	lib/*-client.ts	Cerebro (orquestrador), ticket-upsert loopback, Cloudinary, Twilio

3. Stack tecnologica

3.1 Backend

Pacote	Versao	Papel no sistema
next	16.1.6	Framework web; App Router para paginas e API, servido por um HTTP server customizado
react / react-dom	19.2.3	Renderizacao das paginas Next
socket.io	4.8.3	Broadcast de eventos em tempo real para o inbox
whatsapp-web.js	1.34.6	Canal WhatsApp nao oficial via QR Code (Puppeteer headless)
twilio	5.12.2	Canal WhatsApp oficial: webhook de entrada e API de envio
@supabase/supabase-js	2.48.0	Cliente usado sobre o shim de PostgreSQL
pg	8.20.0	Driver PostgreSQL usado pelo shim postgres-supabase-shim
@prisma/client / prisma	6.19.3	Schema declarado; nao usado em runtime (ver secao de divida tecnica)
firebase-admin	13.6.1	Backend Firestore alternativo (DB_PROVIDER=firestore)
bullmq + ioredis	5.69.3 / 5.9.3	Filas assincronas sobre Redis
cloudinary	2.9.0	Armazenamento de midia (imagens, audio, documentos)
jose	6.1.3	Assinatura e verificacao do JWT de sessao (HS256)
bcryptjs	3.0.3	Hash de senha (custo 10)
zod	4.3.6	Validacao de payload em todas as rotas de escrita
pino / pino-pretty	10.3.1 / 13.1.3	Log estruturado
qrcode	1.5.4	Converte o QR do WhatsApp em data URL para exibir no painel
date-fns	4.1.0	Formatacao de datas em pt-BR

3.2 Frontend (SPA ativa)

Pacote	Versao	Papel
vite	6.2.0	Bundler e dev server na porta 4001, com proxy para a API
react / react-dom	19.2.4	UI
react-router-dom	6.28.0	Roteamento client-side (/inbox, /dashboard, /vault, /contacts, /painel, /admin/*)
tailwindcss	3.4.15	Estilizacao utilitaria com suporte a tema claro/escuro
recharts	3.7.0	Graficos da tela de Metricas
socket.io-client	4.8.3	Assinatura dos eventos em tempo real
@google/genai	1.42.0	Gemini para sugestao de resposta e resumo de conversa (assistencia ao atendente)

A SPA foi originada de um projeto do Google AI Studio (o nome interno no package.json ainda e omnidesk-pro---enterprise-helpdesk) e depois cabeada na API real do WillTalk. Isso explica por que algumas telas ja consomem a API e outras ainda leem services/mockData.ts.

4. Estrutura do repositorio

```
willtalk/  
|- server.cjs           Bootstrap: Next + HTTP server + Socket.IO (porta 4002)  
|- worker.mjs           Workers BullMQ (sla-events, webhook-events, cloudinary-cleanup)  
|- app/  
|  |- layout.tsx        Layout raiz (fontes Manrope + IBM Plex Mono)  
|  |- page.tsx, login/, dashboard/, dashboard/queues/  
|  |- globals.css  
|  \- api/              26 route handlers  
|     |- auth/login, auth/logout, me  
|     |- conversations/ (+ [id])/assign, close, messages, messages/upload, typing)  
|     |- contacts/ (+ [id], [id])/start-conversation)  
|     |- queues/ (+ [id])    quick-replies/ (+ [id])  
|     |- admin/users/ (+ [id])  dashboard/metrics    media/signed  
|     |- whatsapp/status, connect, disconnect  
|     \- webhooks/twilio, webhooks/n8n/ticket-upsert, webhooks/cerebro/reply  
|- components/  
|- lib/  
|  \- services/  
|- frontend/  
|  |- App.tsx, constants.tsx, types.ts  
|  |- components/  
|  |  \- Admin/  
|  |  \- services/  
|- prisma/schema.prisma  Schema declarativo (referencia; nao usado em runtime)  
|- scripts/  
|- docs/  
|- .webjs_auth/  
|- *.bat
```

4.1 Modulos de lib/ por responsabilidade

Modulo	Tam.	Responsabilidade
firestore-repo.ts	50 KB	Implementacao completa da persistencia sobre Firestore
supabase-repo.ts	48 KB	Implementacao completa da persistencia sobre PostgreSQL (~36 funcoes exportadas)
whatsapp-client.ts	29 KB	Ciclo de vida do cliente QR, ingestao inbound/outbound, rate limit, triagem legada
willtalk-webhook.ts	12 KB	Webhook de saida: consolidacao de historico, retry, cooldown, deduplicacao de log
cerebro-auto-reply.ts	11 KB	Resposta assistida vinda do Cerebro: valida origem, telefone e duplicidade
postgres-supabase-shim.ts	8.7 KB	Traduz a API fluente do supabase-js para SQL direto via node-postgres
image-vision.ts	4.7 KB	Analise de print de tela por LLM multimodal (Groq / xAI / OpenAI)
cerebro-orchestrator-client.ts	4.3 KB	Cliente HTTP do orquestrador do Cerebro
utils.ts	3.7 KB	Textos do bot, saudacao por fuso de Brasilia, normalizacao de telefone
n8n-ticket-upsert-client.ts	3.7 KB	Chamada loopback para o ticket-upsert com fallback de porta
repo.ts	3.4 KB	Seletor de backend de dados em tempo de import
investigation-reply.ts	2.5 KB	Roteiro de investigacao pos-triagem (2 rodadas antes do humano)
quick-reply-service.ts	2.3 KB	Substituicao de variaveis nas respostas rapidas
business-hours.ts	2.1 KB	Horario comercial com fuso America/Sao_Paulo e agenda padrao

Modulo	Tam.	Responsabilidade
schemas.ts / auth.ts / api.ts	~4 KB	Zod, JWT de sessao, guardas requireSession e requireRole
realtime.ts / logger.ts / queues.ts	~1.3 KB	Emissao Socket.IO, pino, definicao das filas BullMQ

5. Modelo de dados

O sistema opera hoje sobre PostgreSQL. O schema autoritativo em runtime é `scripts/supabase-schema.sql` — ele é mais recente e mais completo que `prisma/schema.prisma`, que permanece no repositório como documentação do modelo relacional mas não gera cliente usado em produção.

5.1 Entidades e relacionamentos

```
organizations (raiz multi-tenant)
|
+-- users          (admin | gestor | atendente, senha em bcrypt)
+-- channels        (numero Twilio -> organizacao, para roteamento multi-tenant)
+-- queues          (demandas: nome, opcao de menu, cor, SLA padrao)
+-- contacts        (telefone unico por organizacao, avatar, bloqueio, nota interna)
|
|   |
|   +-- conversations (status, triage_completed, menu_attempts, contact_phone)
|   |
|   |   |
|   |   +-- messages (inbound/outbound, text|image|document|audio, midia, author_id)
|   |   +-- tickets  (1:1 com a conversa: fila, responsavel, SLA, nota de satisfacao)
|   |
|   +-- quick_replies (respostas prontas com variaveis)
+-- business_hours   (agenda por dia da semana, com fuso)
|-- audit_logs       (trilha de auditoria com metadados em JSONB)
```

5.2 Tabela: conversations

Coluna	Tipo	Significado operacional
id	TEXT PK	Identificador da conversa (também usado como referência de ticket em vários fluxos)
organization_id	TEXT FK	Tenant
contact_id / contact_phone	TEXT	Contato e telefone desnormalizado para envio rápido
queue_id	TEXT FK	Fila/demanda classificada pela triagem (nulo enquanto não classificado)
status	ENUM	aguardando em_atendimento pendente_cliente encerrado
trriage_completed	BOOLEAN	Marca o fim da fase de bot; depois disso o humano conduz
menu_attempts	INTEGER	Contador duplo: tentativas de menu inválidas e rodadas de investigação da IA
closed_at	TIMESTAMPZ	Momento do encerramento

5.3 Tabela: tickets

Coluna	Tipo	Significado operacional
conversation_id	TEXT UNIQUE	Relação 1:1 com a conversa
queue_id / assignee_id	TEXT FK	Fila e atendente responsável
first_response_due_at	TIMESTAMPZ	Prazo de SLA calculado na classificação (agora + default_sla_mins)
first_response_at	TIMESTAMPZ	Primeira resposta humana efetiva; base do tempo médio de resposta
satisfaction_score	INTEGER	Nota de 1 a 5 respondida pelo cliente no WhatsApp após o encerramento
satisfaction_rated_at	TIMESTAMPZ	Momento da avaliação (não é sobrescrita se já existir)

Coluna	Tipo	Significado operacional
close_reason	TEXT	Motivo informado pelo atendente ao encerrar

5.4 Índices relevantes

Índice	Finalidade
idx_contacts_org_phone (UNIQUE)	Garante um contato por telefone por organização — base da reutilização de conversa
idx_tickets_conversation (UNIQUE)	Reforça a cardinalidade 1:1 entre conversa e ticket
idx_messages_external_id (parcial)	Deduplicação de mensagem por ID externo (MessageSid do Twilio, id do whatsapp-web.js, event_id do n8n)
idx_messages_conversation	Carregamento do histórico ordenado por data decrescente
idx_conversations_org_status	Listagem do inbox filtrada por status
idx_business_hours_org_day (UNIQUE)	Uma janela de expediente por dia da semana

5.5 Camada de acesso a dados

`lib/repo.ts` não é um ORM: é um **seletor de implementação em tempo de import**. Ele lê `DB_PROVIDER` e, para cada função exportada, escolhe entre `supabase-repo` (valores `supabase`, `postgres`, `postgresql`) e `firestore-repo` (padrão). Todas as demais camadas importam apenas de `@/lib/repo` e desconhecem o banco real.

```
const provider = (process.env.DB_PROVIDER || "firestore").toLowerCase();
const useSupabase = ["supabase", "postgres", "postgresql"].includes(provider);

function pick(key) {
  if (useSupabase && key in supabaseRepo) return supabaseRepo[key];
  return firestoreRepo[key];
}

export const listConversations = pick("listConversations");
// ... 36 exports seguem o mesmo padrão
```

Abaixo do `supabase-repo` existe ainda o `postgres-supabase-shim.ts`, que reimplementa a API fluente do `supabase-js` (`from().select().eq().maybeSingle()`) sobre SQL gerado para o driver pg. Ou seja: o código é escrito no estilo Supabase, mas executa contra um PostgreSQL local comum.

6. Autenticacao, sessao e permissoes

6.1 Fluxo de login

1. A SPA envia POST `/api/auth/login` com e-mail e senha (validados por Zod: e-mail valido, senha com no minimo 6 caracteres).
2. A rota busca o usuario por e-mail e rejeita imediatamente se `isActive === false`.
3. A senha e comparada com o hash bcrypt armazenado.
4. Um JWT HS256 e assinado com `JWT_SECRET` contendo `userId`, `organizationId`, `role`, `name` e `email`, com validade de **12 horas**.
5. O token e gravado no cookie `willtalk_session` — `httpOnly`, `sameSite lax`, `path /`, `secure` apenas em producao.
6. A SPA chama GET `/api/me` para hidratar a sessao e mapear o papel do backend para o enum da UI.

6.2 Guardas do servidor

```
// lib/api.ts
requireSession() -> 401 { error: "Nao autenticado" } se nao houver cookie valido
requireRole(["admin", "gestor"], session.role) -> 403 { error: "Sem permissao" }
```

Toda rota autenticada comeca com `requireSession()` e usa `session.organizationId` como filtro obrigatorio nas consultas — o isolamento multi-tenant e aplicado na camada de rota, nao no banco (nao ha RLS).

6.3 Matriz de papeis

Recurso	admin	gestor	atendente
Inbox, enviar mensagem, assumir e encerrar chamado	Sim	Sim	Sim
Contatos e Cofre de Acesso	Sim	Sim	Sim
Respostas rapidas (criar)	Sim	Sim	Sim
Metricas / Dashboard	Sim	Sim	Nao (redireciona para o inbox)
Painel de QR do WhatsApp (conectar/desconectar)	Sim	Sim	Nao
Gestao de equipe (criar/editar/desativar usuarios)	Sim	Sim (API) / Nao (menu)	Nao
Tipos de ticket (filas) e admin de respostas rapidas	Sim	Nao (menu)	Nao

Existe uma **divergencia entre a permissao da UI e a do backend**: o menu lateral restringe 'Equipe', 'Tipos de Ticket' e 'Respostas Rapidas' a *admin*, mas as rotas correspondentes aceitam *admin* e *gestor*. Como a autorizacao real e a do backend, um gestor consegue chamar essas APIs diretamente. Isso e intencional ou uma inconsistencia a decidir.

6.4 Autenticacao maquina-a-maquina

Rota	Mecanismo
POST <code>/api/webhooks/n8n/ticket-upsert</code>	Bearer igual a <code>WILLTALK_WEBHOOK_TOKEN</code> . Ha um bypass opcional para <code>127.0.0.1/localhost</code> via <code>WILLTALK_ALLOW_LOCAL_UPSERT_NO_AUTH=true</code>
POST <code>/api/webhooks/cerebro/reply</code>	Tres verificacoes em cadeia: recurso habilitado, caminho exatamente igual ao configurado, e Bearer valido. Alem disso o corpo precisa trazer origem igual a <code>WILLTALK_AUTO_REPLY_SOURCE</code>

Rota	Mecanismo
POST /api/webhooks/twilio	Validacao criptografica da assinatura X-Twilio-Signature com o auth token da conta
Saida: WILLTALK_WEBHOOK_URL	Envia Authorization: Bearer e o cabeçalho x-willtalk-event

7. Canais de WhatsApp

A variável `WHATSAPP_PROVIDER` define o canal ativo. O valor em uso hoje é **unofficial**.

7.1 Modo unofficial (whatsapp-web.js + QR Code)

Um cliente Puppeteer headless mantém uma sessão de WhatsApp Web autenticada. A sessão é persistida em disco por LocalAuth na pasta `.webjs_auth/`, então reinícios do servidor não exigem novo escaneamento de QR.

7.1.1 Máquina de estados

Estado	Significado	Exposto em
idle	Cliente nunca inicializado nesta execução	GET /api/whatsapp/status
initializing	Puppeteer subindo o navegador	idem
qr	QR Code disponível como data URL para escanear	idem (campo qrDataUrl)
ready	Sessão autenticada; mensagens fluem	idem (campo connectedPhone)
disconnected	Sessão caiu; motivo em <code>lastError</code>	idem
error	Falha de autenticação ou de inicialização	idem

7.1.2 Proteções implementadas na ingestão

- **Filtro de ruído:** descarta status, broadcast e nove tipos internos do protocolo (`e2e_notification`, `notification`, `protocol`, `gp2`, `revoked`, `ciphertext`, `broadcast_notification`, `reaction`, `call_log`).
- **Somente conversa 1:1:** aceita apenas chatIds terminados em `@c.us` ou `@lid`.
- **Descarte de fila offline:** ao reconectar, o `whatsapp-web.js` reemite todo o histórico pendente. O sistema registra o instante em que ficou pronto (`clientReadyAt`) e ignora qualquer mensagem com timestamp anterior — sem isso o bot responderia a todas as mensagens antigas de uma vez.
- **Rate limit de envio:** intervalo mínimo de 1500 ms entre mensagens (`WA_MIN_SEND_INTERVAL_MS`), para reduzir risco de bloqueio pelo WhatsApp.
- **Marcacao de envio automático:** mensagens do bot são registradas em um conjunto `recentBotSends` antes do envio, para que o eco do evento `message_create` não mude indevidamente o status da conversa para `em_atendimento`.
- **Supressão de ruído conhecido:** um guard global de `unhandledRejection` silencia dois erros recorrentes e inofensivos do Puppeteer no Windows.

7.1.3 Captura de mensagens enviadas pelo celular

O handler de `message_create` persiste também as mensagens que o atendente envia direto do aparelho conectado, criando contato e conversa se necessário. Assim, uma thread iniciada pelo celular aparece no inbox. A lógica preserva o status: conversas novas ficam em `aguardando`, conversas encerradas **não** são reabertas, e apenas uma mensagem humana real promove a conversa para `em_atendimento`.

7.2 Modo twilio (oficial)

- Entrada: POST `/api/webhooks/twilio` em formato `application/x-www-form-urlencoded`, com validação de assinatura.
- Callbacks de status (`MessageStatus`) são respondidos com TwiML vazio, sem efeito colateral.

- Roteamento multi-tenant: o número de destino (T_0) é resolvido para uma organização via tabela `channels`; sem correspondência, cai na organização padrão.
- Mídia recebida e baixada com autenticação básica da Twilio e reenviada ao Cloudinary.
- Com `WILLTALK_N8N_ONLY=true`, o webhook não responde TwiML de menu: apenas delega ao ticket-upsert e devolve TwiML vazio.

7.3 Estratégia de envio com fallback

A função `sendTriageMessageToWhatsApp` concentra o envio automático e implementa uma cadeia de tentativas:

1. Se o provider não for *unofficial*, envia direto pela Twilio.
2. Caso contrário, inicializa o cliente QR e aguarda até 12 segundos pelo estado *ready* (`WILLTALK_TRIAGE_READY_WAIT_MS`).
3. Se ficar pronto, envia por `whatsapp-web.js`.
4. Se falhar ou não ficar pronto, tenta a Twilio como fallback — salvo se `WILLTALK_TRIAGE_TWILIO_FALLBACK=false`.
5. Se ambos falharem, lança erro com mensagem acionável; o log inclui a dica de conectar o QR no painel.

Existe ainda o modo **dry run** (`WILLTALK_DRY_RUN_WHATSAPP=true`): as respostas automáticas são persistidas no histórico como outbound com `id dryrun-<timestamp>`, mas nada é enviado ao cliente real. É o modo recomendado para testar a triagem sem WhatsApp conectado.

8. Motor de triagem

Esta é a área mais densa do sistema e a que mais evoluiu. Existem **tres motores de triagem coexistindo**, selecionados por variáveis de ambiente. Entender qual está ativo é o passo número um em qualquer diagnóstico.

8.1 Seleção do motor

Condição	Motor ativo
WILLTALK_AI_TRIAGE_ONLY diferente de <i>false</i> (padrão) e CEREBRO_ORCHESTRATOR_URL preenchida	Motor A — Orquestrador do Cerebro decide tudo
Triagem por IA ativa, mas o Cerebro não respondeu ou não está configurado	Motor B — IA local com visão computacional e heurísticas
WILLTALK_AI_TRIAGE_ONLY=false e WILLTALK_N8N_ONLY desligado	Motor C — menu numérico legado dentro do whatsapp-client / webhook Twilio

8.2 Motor A — Orquestrador do Cerebro Operacional

O WillTalk monta um pacote de contexto e faz um POST para {CEREBRO_ORCHESTRATOR_URL}/api/orquestrador/v1/mensagem (a URL pode ser informada como base ou como endpoint completo). O timeout é de 18 segundos.

8.2.1 Requisição

Cabecalhos:

```
Authorization: Bearer <CEREBRO_ORCHESTRATOR_TOKEN | CEREBRO_INGEST_TOKEN>
X-Source-System: willtalk
X-Tenant-Id: <organization_id>
X-Ingestion-Id: <event_id>
X-Source-Entity-Id: <conversation_id>
```

Corpo:

```
{
  "platform": "willtalk",
  "organization_id": "...", "event_id": "...", "conversation_id": "...",
  "cliente": { "nome": "...", "telefone": "whatsapp:+55..." },
  "mensagem": "texto do cliente",
  "media_url": "https://res.cloudinary.com/...", // opcional
  "mime_type": "image/jpeg", // opcional
  "business_hours_open": true,
  "conversation": { "triage_completed": false, "menu_attempts": 0, "queue_id": null },
  "queues": [ { "id", "menu_option", "name", "default_sla_mins", "is_active" } ]
}
```

8.2.2 Resposta esperada

```
{
  "reply_text": "texto que será enviado ao cliente",
  "triage_completed": true,
  "menu_attempts": 1,
  "queue_id": "<id da fila classificada ou null>",
  "reason": "classificado_por_intencao"
}
```

O WillTalk então aplica a decisão: se a fila mudou, grava o ticket com a nova fila e recalcula *first_response_due_at* como *agora* + *default_sla_mins*; atualiza *triage_completed* e *menu_attempts* na conversa; e envia *reply_text* pelo WhatsApp.

Degradacao resiliente: mesmo quando o Cerebro retorna HTTP de erro, se o corpo contiver um `reply_text` util, o WillTalk usa esse texto em vez de cair no fallback. Se nao houver texto, o cliente retorna `null` e o Motor B assume, sem interromper o atendimento.

8.3 Motor B — IA local com fallback em camadas

Quando o orquestrador nao produz resposta, o ticket-upsert monta a resposta localmente, em ordem de prioridade:

1. **Resposta pronta no metadata:** se o chamador enviou `metadata.reply_text` (ou `ai_reply_text`, ou `resposta_sugerida`), ela e usada.
2. **Analise de imagem:** se veio midia e ela parece imagem, chama um LLM multimodal com um prompt de analista de suporte que pede: o que foi identificado, de 2 a 4 passos objetivos e pedido do erro literal.
3. **Heurísticas por palavra-chave:** reconhece quatro dominios recorrentes — banco de dados/PostgreSQL, fiscal/nota/SEFAZ, impressora/etiqueta e lentidao/travamento — e devolve um roteiro pratico especifico.
4. **Texto generico:** se nada acima se aplica, pede o modulo afetado e o grau do impacto.

Se o horario estiver fora do expediente e `WILLTALK_AI_APPEND_OUT_OF_HOURS` estiver ligado, um aviso de fora de horario e anexado ao final da resposta.

8.3.1 Provedores de visao computacional

Base detectada	Modelo padrao	Chave de API buscada (em ordem)
api.groq.com	meta-llama/llama-4-scout-17b-16e-instruct	VISION_API_KEY, XAI_API_KEY, WILLTALK_VISION_API_KEY, GROK_API_KEY, AI_API_KEY
api.openai.com	gpt-4o-mini	mesma cadeia, com preferencia por OPENAI_API_KEY
api.x.ai (padrao)	grok-2-vision-1212	mesma cadeia, com preferencia por XAI_API_KEY

Limites aplicados: temperatura 0.2, teto de 280 tokens de saida, timeout de 45 segundos e a resposta e truncada em 450 caracteres antes de ir para o cliente.

8.4 Motor C — menu numerico legado

Implementado de forma quase identica em dois lugares: `lib/whatsapp-client.ts` e `app/api/webhooks/twilio/route.ts`. Roteiro:

1. Primeira mensagem sem fila definida: envia o menu de demandas ativas, com saudacao correta pelo fuso de Brasilia (Bom dia / Boa tarde / Boa noite) e o nome do cliente.
2. Numero valido: grava a fila na conversa e no ticket, calcula o prazo de SLA, zera as tentativas e pede evidencias (print da tela, erro literal, impacto total ou parcial).
3. Numero invalido: reenvia o menu com o contador *Tentativa N/3*.
4. Na terceira tentativa invalida: marca `triage_completed = true` e encaminha para atendimento humano.
5. Com fila ja definida: entram **2 rodadas de investigacao assistida** (`INVESTIGATION_AI_ROUNDS = 2`) — a primeira analisa a imagem se houver, as seguintes aplicam heurísticas — e depois encaminha ao tecnico da fila.
6. Sem filas cadastradas: registra o chamado e informa que um atendente respondera.

Apos `triage_completed = true`, o bot **nao envia mais nada** no fluxo legado — quem responde e o atendente pelo painel. Conversas encerradas que recebem nova mensagem sao reabertas silenciosamente como *aguardando*, sem mensagem automatica, para evitar spam.

8.5 Guardas anti-loop e deduplicacao

Guarda	Janela	O que evita
Deduplicacao por external_id	permanente	Reprocessar a mesma mensagem (MessageSid, id do WhatsApp, event_id)
Janela de payload identico	8 segundos	Retries do n8n que regeneram o event_id mas repetem o conteudo
Guarda de prompt de triagem repetido	45 segundos	Reenviar o mesmo pedido de classificacao quando o cliente manda varias mensagens seguidas
Deduplicacao da resposta do Cerebro	5 minutos	Enviar duas vezes a mesma sugestao (hash SHA-256 do texto por ticket)
Supressao de webhook no ticket-upsert	por padrao	Ciclo infinito WillTalk -> n8n -> ticket-upsert -> WillTalk
Cooldown de endpoint com falha	5 segundos	Martelar um webhook fora do ar e inundar o log

A ingestao direta do WhatsApp nao oficial marca `metadata.ingest_origin = "whatsapp-unofficial"` e com isso **pula a janela de 8 segundos** — porque ali repeticoes legitimas do cliente sao esperadas.

Limitacao conhecida: todos esses guardas usam Map e Set em memoria do processo. Eles nao sobrevivem a um restart e nao sao compartilhados entre instancias. Em um cenario com mais de um no, a deduplicacao passaria a depender apenas do `external_id` no banco.

9. Fluxos de ponta a ponta

9.1 Fluxo 1 — Mensagem nova de cliente ate a resposta do bot

1. Cliente envia mensagem no WhatsApp
2. whatsapp-web.js dispara o evento "message"
3. Filtros: não é fromMe, e chat 1:1, não é ruído, timestamp posterior ao ready
4. Se há mídia: download -> upload no Cloudinary -> secure_url
5. handleInboundViaBotTriagem monta o payload e chama, por loopback:
POST http://127.0.0.1:4002/api/webhooks/n8n/ticket-upsert
Authorization: Bearer <WILLTALK_WEBHOOK_TOKEN>
metadata: { ingest_origin: "whatsapp-unofficial" }
6. ticket-upsert:
 - a) valida o Bearer e o schema Zod
 - b) normaliza o telefone para whatsapp:+55DDDDNUMERO
 - c) checa deduplicação (external_id e janela de payload)
 - d) getOrCreateContactAndOpenConversation -> contato + conversa + ticket
 - e) persiste a mensagem inbound
 - f) emite message.created e conversation.created/updated via Socket.IO
 - g) grava audit_log (ação n8n_ticket_upsert)
 - h) consulta o horário comercial e as filas ativas
 - i) decide a resposta (Motor A -> B)
 - j) envia pelo WhatsApp e persiste como outbound
 - k) se a triagem acabou de ser concluída, dispara ingestão no Cerebro (fire-and-forget, timeout de 10s, não bloqueia)
7. A SPA recebe os eventos e atualiza o inbox sem recarregar

9.2 Fluxo 2 — Atendente responde pelo painel

1. O atendente clica em **Assumir**: POST /api/conversations/:id/assign grava o responsável, registra auditoria, emite *conversation.updated* e dispara o webhook *ticket_updated*.
2. Ao enviar texto, POST /api/conversations/:id/messages resolve as variáveis de resposta rápida ({cliente}, {saudacao}, {primeiro_nome}, {ticket}, {data}, {hora}, {user}, {empresa}).
3. A mensagem sai no WhatsApp com o nome do atendente na primeira linha, seguida do texto.
4. **Decisão de projeto relevante**: se o envio externo falhar, a mensagem ainda é persistida no histórico e o erro fica apenas no log — o atendente não perde o que escreveu.
5. São emitidos *message.created* e *conversation.updated*, além dos webhooks *message_sent* e *ticket_updated*.

9.3 Fluxo 3 — Encerramento e pesquisa de satisfação

1. POST /api/conversations/:id/close exige um motivo com pelo menos 3 caracteres.
2. TicketService.close levanta os public_ids do Cloudinary da conversa e **enfileira a limpeza em background** — falha de Redis ou Cloudinary nunca bloqueia o encerramento.
3. A conversa vai para *encerrado*, o ticket recebe motivo e data de fechamento, e a auditoria é gravada.
4. Se sendSurvey foi marcado, o cliente recebe: *Seu atendimento com <atendente> foi finalizado. De 1 a 5, qual nota voce da para o atendimento?* — enviado em background com fromBot: true, para não reabrir a conversa.
5. Quando o cliente responde um dígito de 1 a 5, recordSatisfactionRatingByPhone localiza a última conversa encerrada daquele telefone e grava a nota no ticket. Notas já existentes **não** são sobrescritas.

9.4 Fluxo 4 — Resposta assistida vinda do Cerebro

1. O Cerebro (direto ou via n8n) chama POST /api/webhooks/cerebro/reply.

2. Cadeia de validacao: recurso habilitado (503 se nao), caminho permitido (404), Bearer valido (401), schema Zod (400), origem correta (400), canal igual a whatsapp (400).
3. Deduplicacao por hash da sugestao dentro de 5 minutos (200 com *duplicate_ignored*).
4. Resolucao do alvo: casamento estrito por id de conversa ou de ticket; se houver ambiguidade, retorna 404 em vez de arriscar o destinatario errado.
5. **Trava de seguranca:** se `cliente_telefone` vier no payload e nao casar com o telefone do contato, retorna **409** — impede responder no numero errado.
6. A mensagem final e envelopada com um aviso de sugestao automatica e o convite a responder para acionar um tecnico.
7. Tudo e registrado em `audit_log` com o status recebido / enviado / duplicado / erro.

10. Referência da API

Todas as rotas abaixo vivem em `app/api/` e são servidas em `http://localhost:4002`. Salvo indicação contrária, exigem o cookie de sessão e retornam 401 sem ele.

10.1 Autenticação e sessão

Método e rota	Auth	Descrição
POST <code>/api/auth/login</code>	publica	E-mail e senha; grava o cookie de sessão
POST <code>/api/auth/logout</code>	publica	Limpa o cookie
GET <code>/api/me</code>	cookie	Devolve o payload da sessão (ou user: null com status 200)

10.2 Conversas e mensagens

Método e rota	Papel	Descrição
GET <code>/api/conversations</code>	qualquer	Lista as conversas do tenant; aceita <code>?status=aguardando em_atendimento pendente_cliente encerrado</code>
POST <code>/api/conversations/:id/assign</code>	qualquer	Assume o chamado para o usuário da sessão
POST <code>/api/conversations/:id/messages</code>	qualquer	Envia texto; resolve variáveis de resposta rápida e assina com o nome do atendente
POST <code>/api/conversations/:id/messages/upload</code>	qualquer	Envia mídia (upload no Cloudinary)
POST <code>/api/conversations/:id/close</code>	qualquer	Encerra com motivo; opcionalmente dispara a pesquisa de satisfação
POST <code>/api/conversations/:id/typing</code>	qualquer	Propaga o indicador de digitação ao contato no WhatsApp

10.3 Cadastros e configuração

Método e rota	Papel	Descrição
GET / POST <code>/api/queues</code>	sessão	Lista e cria filas (demandas): nome, opção de menu 1-99, cor #RRGGBB, SLA de 5 a 1440 min
PATCH <code>/api/queues/:id</code>	sessão	Atualiza a fila (inclusive ativar/desativar)
GET / POST <code>/api/quick-replies</code>	todos	Lista e cria respostas rápidas com categoria opcional
PATCH / DELETE <code>/api/quick-replies/:id</code>	todos	Edita e remove
GET <code>/api/contacts</code>	sessão	Lista contatos com dados agregados de conversa
PATCH <code>/api/contacts/:id</code>	sessão	Nome, telefone, bloqueio e nota interna (até 2000 caracteres)
POST <code>/api/contacts/:id/start-conversation</code>	sessão	Abre conversa a partir do contato (atendimento ativo)
GET / POST <code>/api/admin/users</code>	admin, gestor	Lista ativos e cria usuário (senha em bcrypt, 409 se o e-mail existir)
PATCH / DELETE <code>/api/admin/users/:id</code>	admin, gestor	Edita e desativa (soft delete)

10.4 Operação e integrações

Método e rota	Auth	Descrição
GET /api/dashboard/metrics	sessao	Totais por status, tempo médio de primeira resposta, volume por demanda e média de satisfação
GET /api/whatsapp/status	sessao	Provider ativo e estado completo do cliente (inclui o QR em data URL)
POST /api/whatsapp/connect	admin, gestor	Inicializa o cliente e gera o QR
POST /api/whatsapp/disconnect	admin, gestor	Destroi a sessão
GET /api/media/signed?publicId=	sessao	Redireciona 302 para a URL assinada do Cloudinary
POST /api/webhooks/twilio	assinatura	Ingestão Twilio
POST /api/webhooks/n8n/ticket-upsert	Bearer	Ponto central de ingestão e triagem
POST /api/webhooks/cerebro/reply	Bearer	Resposta assistida do Cerebro Operacional

10.5 Contrato do ticket-upsert

10.5.1 Requisição

POST /api/webhooks/n8n/ticket-upsert
 Authorization: Bearer <WILLTALK_WEBHOOK_TOKEN>
 Content-Type: application/json

```
{
  "event_id": "evt-001", // obrigatorio, chave de deduplicacao
  "canal": "whatsapp", // obrigatorio, literal
  "organization_id": "org...", // opcional; cai em DEFAULT_ORG_ID
  "cliente": { "nome": "Cliente Teste", "telefone": "5511999999999" },
  "mensagem": "Preciso abrir chamado", // ou "body"; obrigatorio se nao houver midia
  "media_url": "https://...", // ou "mediaUrl"
  "mime_type": "image/png", // ou "mimeType"
  "queue_id": "opcional",
  "prioridade": "baixa|media|alta|critica",
  "ticket_ref": "opcional", // ou "ticket_id"
  "metadata": { "origem_fluxo": "n8n-onboarding-v1" }
}
```

10.5.2 Resposta

```
{
  "action": "created | updated | ignored_duplicate | human_handoff | error",
  "conversationStatus": "aguardando",
  "triageCompleted": false,
  "menuAttempts": 0,
  "queueId": null,
  "shouldReply": true,
  "replyText": "texto enviado ao cliente",
  "events": ["message.created", "conversation.created"],
  "webhooks": ["webhooks_suprimidos_no_ticket_upsert"],
  "reason": "inbound_processado_com_regras_de_identidade_triagem_e_seguranca",
  "organizationId": "...", "contactId": "...", "conversationId": "...",
  "ticketId": "...", "duplicate": false, "created": true
}
```

Código	Quando ocorre
200	Processado, ou duplicata ignorada (com duplicate: true)

Codigo	Quando ocorre
400	JSON invalido, schema reprovado, telefone invalido, ou nem mensagem nem midia
401	Bearer ausente ou incorreto (e sem o bypass local habilitado)
409	ticket_ref ambiguo — exige revisao humana em vez de adivinhar a conversa

11. Integrações externas

11.1 Webhook de saída (WillTalk para n8n / Cerebro)

Disparado por `sendWillTalkWebhook` em pontos de mudança de estado. Antes de enviar, o módulo carrega um *snapshot* da conversa e consolida todo o histórico em um único campo de texto.

11.1.1 Payload

```
POST <WILLTALK_WEBHOOK_URL>
Authorization: Bearer <WILLTALK_WEBHOOK_TOKEN>
x-willtalk-event: ticket_created | ticket_updated | message_received | message_sent

{
  "ticket_id": "...",
  "cliente": "Nome do cliente",
  "cliente_telefone": "5511999999999", // apenas digitos
  "mensagens": "2026-07-22T18:30:00.000Z | cliente: texto\n... | atendente: texto",
  "canal": "whatsapp",
  "tecnico": "Nome do atendente",
  "data_evento": "2026-07-22T18:30:00.000Z"
}
```

11.1.2 Tratamento do histórico

- Cada linha segue o formato `<ISO timestamp> | <ator>: <conteudo>`.
- Um filtro de ruído remove oito padrões de mensagem automática (menu, tentativa N/3, pesquisa de satisfação, aviso de fora de horário etc.), para não poluir o contexto do Cerebro.
- O texto é truncado **a partir das mensagens mais antigas**, respeitando `WILLTALK_WEBHOOK_MAX_CHARS` (padrão 12000), e recebe o marcador `[histórico truncado: N mensagens antigas removidas]`.
- Retry com backoff exponencial (400 ms, 800 ms, 1600 ms), até 3 tentativas por padrão, timeout de 8 s cada.
- Erros de conexão ativam um cooldown de 5 s no endpoint, e os logs de falha são agregados para não inundar o pino.

O ticket-upsert **suprime** os webhooks de saída por padrão (`WILLTALK_TICKET_UPSERT_EMIT_WEBHOOKS`), retornando o marcador `webhooks_suprimidos_no_ticket_upsert`. Isso é proposital: reemitir dali criaria o ciclo WillTalk -> n8n -> ticket-upsert -> WillTalk.

11.1.3 Proteção contra eco de histórico

Se o próprio texto consolidado voltar como mensagem de entrada (situação real quando o n8n reencaminha o payload), a função `normalizeInboundEchoPayload` detecta o marcador de truncamento ou tamanho acima de 2000 caracteres, extrai a **última fala do cliente** e a armazena prefixada com `[eco de histórico omitido]`. Sem isso, uma mensagem gigante quebraria a triagem e o inbox.

11.2 Configuração do n8n

1. No primeiro node **Webhook** do workflow: método POST e path `willtalk-ingestao` (apenas o slug, sem `/webhook/`).
2. Clicar em *Listen for test event* ou salvar, e depois **ativar** o workflow — somente workflow ativo registra a URL de produção.
3. Verificar com `scripts/verify-n8n-webhook.ps1` — precisa retornar HTTP 200, não 404.
4. Se o n8n gerou um path com UUID, ou se ajusta o path no node, ou se aponta `WILLTALK_WEBHOOK_URL` para a Production URL exibida.

11.3 Cloudinary

Aspecto	Comportamento
Pasta	willtalk/messages
Tipo de recurso	image para imagens, video para audio (convencao do Cloudinary), raw para os demais
Origem Twilio	Download autenticado com Basic Auth da conta antes do upload
Origem QR	Upload direto do base64 devolvido por downloadMedia()
Falha no upload	Nao interrompe o fluxo: segue com a URL original ou sem midia
Acesso	GET /api/media/signed?publicId= gera URL assinada e redireciona
Limpeza	Ao encerrar o chamado, os public_ids sao enfileirados no BullMQ para exclusao

11.4 Gemini na interface do atendente

A SPA embute @google/genai com o modelo gemini-3-flash-preview para duas assistencias: **sugerir resposta** (a partir do assunto e do historico recente) e **resumir a conversa** em tres pontos, util na passagem de plantao. As duas funcoes degradam para uma mensagem de erro amigavel se a API falhar.

Risco de segurança a tratar antes de publicar: o vite.config.ts injeta GEMINI_API_KEY no bundle via define. Em um build de producao, essa chave fica exposta no JavaScript entregue ao navegador. O correto e mover essas chamadas para uma rota do backend.

12. Tempo real

O Socket.IO é anexado ao mesmo servidor HTTP do Next, no caminho `/socket.io`, e exposto globalmente em `global.__io`. O helper `emitRealtime(evento, payload)` faz broadcast para todos os clientes conectados.

Evento	Direção	Quando é emitido / consumido
connected	servidor para cliente	Confirmação de handshake
message.created	servidor para cliente	Nova mensagem inbound ou outbound; a SPA anexa a mensagem localmente e mantém as últimas 50
conversation.created	servidor para cliente	Primeira mensagem de um contato novo
conversation.updated	servidor para cliente	Mudança de status, fila ou responsável
sla.breached	servidor para cliente	Ouvido pela interface, mas nunca emitido pelo backend — SLA não está fechado
typing:start / typing:stop	cliente para servidor	Retransmitidos como typing para os demais clientes

Não há salas (rooms) nem escopo por organização no broadcast. Todo evento vai para todos os sockets conectados, incluindo os de outra organização. Como os payloads carregam ids de conversa e trechos de mensagem, isso é um vazamento potencial em um cenário realmente multi-tenant. A CORS também está em `origin: ""`.

12.1 Estratégia de atualização da interface

A SPA combina duas técnicas: **aplicação otimista local** para `message.created` (anexa a mensagem sem ir ao servidor) e **refresh agendado com debounce** de 300 ms para os eventos de conversa, evitando uma rajada de requisições quando vários eventos chegam juntos. Um *flag* de não autorizado impede loop de requisições após a expiração da sessão, redirecionando para o login.

13. Interfaces de usuário

O sistema tem **duas interfaces**. A que a operação usa e a SPA em frontend/, servida pelo Vite na porta 4001. As páginas Next em app/dashboard são a versão anterior e continuam acessíveis na porta 4002.

13.1 SPA OmniDesk (interface ativa)

Rota	Tela	Estado
/inbox	Inbox de conversas — o coração da operação (~51 KB de lógica)	API real
/dashboard	Métricas com gráficos Recharts	mockTickets
/vault	Cofre de Acesso (credenciais de cliente)	mockRemoteAccess
/contacts	Lista e edição de contatos	API real
/painel	QR Code do WhatsApp, status e conexão (admin e gestor)	API real
/admin/usuarios	Gestão de equipe	API real
/admin/tipos	Tipos de ticket (filas)	API real
/admin/respostas-rapidas	Biblioteca de respostas rápidas	API real

13.1.1 Características da SPA

- Sessão por cookie httpOnly, com espelho em localStorage apenas para hidratação imediata da UI; a autoridade continua sendo GET /api/me.
- Indicador de status do WhatsApp no cabeçalho, com polling a cada 10 segundos.
- Tema claro e escuro via ThemeContext, aplicado em toda a interface.
- Sidebar colapsável com filtragem de itens por papel do usuário.
- Cliente HTTP próprio (services/api.ts) com credentials: 'include' e uma mensagem de erro dedicada quando o backend está fora do ar: *"Backend inacessível. Suba o servidor na raiz (npm run dev) — API na porta 4002."*

13.2 Interface Next legada

components/dashboard-client.tsx entrega uma versão enxuta em três colunas: KPIs e volume por demanda à esquerda, lista de chamados ao centro, chat à direita, mais o bloco de QR do WhatsApp. Funciona e usa a mesma API, mas não recebeu as funcionalidades novas (respostas rápidas, contatos, admin, tema, upload de mídia).

14. Regras de negocio consolidadas

14.1 Horario comercial

Resolvido sempre no fuso America/Sao_Paulo, independentemente do fuso do servidor. O sistema consulta a tabela `business_hours` para o dia da semana; se não houver registro ativo, aplica a agenda padrão embutida no código:

Dia	Expediente padrão
Segunda a sexta	07:00 as 18:00
Sabado	07:00 as 17:00
Domingo	fechado

Fora do expediente, as respostas automáticas recebem o sufixo: *"Estamos fora do horario comercial. Responderemos no proximo expediente."*

14.2 Normalizacao de telefone

O formato canônico interno é `whatsapp:+55DDDNUMERO`. A normalização remove tudo que não for dígito e prefixa 55 quando o número não começa com o código do Brasil. A comparação entre telefones é tolerante: considera iguais se um for sufixo do outro, para absorver as variações do nono dígito.

14.3 Ciclo de vida do status da conversa



Regras que protegem o status:

- . envio do bot (`fromBot`) nunca promove para `em_atendimento`
- . conversa nova nunca abre já em `em_atendimento`
- . conversa encerrada não é reaberta por mensagem do bot (ex.: pesquisa de satisfação)
- . o status `pendente_cliente` existe no modelo, mas não há transição automática para ele

14.4 SLA

- Cada fila define `default_sla_mins` (padrão 30, faixa aceita de 5 a 1440).
- No momento da classificação, o ticket recebe `first_response_due_at` = agora + SLA.
- `first_response_at` alimenta o tempo médio de primeira resposta no dashboard.
- **O que falta:** nenhum job monitora o vencimento. `enqueueSlaCheck` está implementada em `lib/queues.ts` mas **nunca é chamada**, e o worker de `sla-events` apenas escreve no log. O evento `sla.breached` nunca chega à interface.

14.5 Variáveis de resposta rápida

Variavel	Valor substituido
{user}	Nome do atendente logado
{cliente}	Nome completo do contato
{primeiro_nome}	Primeiro nome do contato, com numeros removidos
{empresa}	Nome da empresa (quando informado)
{ticket}	Primeiros 8 caracteres do id da conversa
{data} / {hora}	Data e hora formatadas em pt-BR
{saudacao}	Bom dia (05:00-11:59), Boa tarde (12:00-17:59), Boa noite (18:00-04:59)

15. Configuração e operação

15.1 Variáveis de ambiente

Todas ficam em `.env` na raiz (arquivo ignorado pelo git). Abaixo os nomes e o papel de cada uma — os valores não são reproduzidos aqui.

15.1.1 Núcleo

Variável	Padrão	Função
PORT	4002	Porta do servidor principal
JWT_SECRET	dev_secret_change_me	Assinatura da sessão — obrigatório trocar em produção
DEFAULT_ORG_ID	org_willtalk_default	Tenant usado quando o canal não resolve a organização
LOG_LEVEL	info	Nível do log
DB_PROVIDER	firestore	postgres / supabase / firestore
DATABASE_URL	-	String de conexão PostgreSQL
PG_SSL	-	Liga SSL no driver pg
REDIS_URL	-	Sem ela os workers encerram e as filas ficam desligadas

15.1.2 Canais e mídia

Variável	Função
WHATSAPP_PROVIDER	unofficial (QR) ou twilio
WHATSAPP_SESSION_NAME	clientId do LocalAuth; determina a pasta da sessão
WA_MIN_SEND_INTERVAL_MS	Intervalo mínimo entre envios (padrão 1500)
TWILIO_ACCOUNT_SID / AUTH_TOKEN / WHATSAPP_NUMBER	Credenciais e número de origem da Twilio
CLOUDINARY_CLOUD_NAME / API_KEY / API_SECRET	Credenciais de mídia; sem elas o upload é simplesmente pulado
FIREBASE_PROJECT_ID / CLIENT_EMAIL / PRIVATE_KEY / SERVICE_ACCOUNT_JSON / SERVICE_ACCOUNT_FILE	Credenciais do backend Firestore alternativo

15.1.3 Triagem e integrações

Variável	Padrão	Função
WILLTALK_AI_TRIAGE_ONLY	true	Liga o motor de IA; <i>false</i> volta ao menu numérico legado
WILLTALK_N8N_ONLY	-	Faz o inbound delegar ao ticket-upsert em vez de responder no próprio canal
CEREBRO_ORCHESTRATOR_URL	-	Base ou endpoint do orquestrador; vazio desliga o Motor A
CEREBRO_ORCHESTRATOR_TOKEN / CEREBRO_INGEST_TOKEN	-	Bearer do Cerebro
CEREBRO_INGESTAO_URL	http://127.0.0.1:3000/api/ingestao/willtalk	Ingestão disparada quando a triagem conclui

Variavel	Padrao	Funcao
CEREBRO_BASE_URL	-	Referencia para montar URLs nos nodes do n8n
WILLTALK_AI_APPEND_OUT_OF_HOURS	true	Anexa aviso de fora de horario
WILLTALK_DRY_RUN_WHATSAPP	-	Simula envios; persiste como outbound sem enviar
WILLTALK_TRIAGE_TWILIO_FALLBACK	true	Permite cair na Twilio quando o QR nao esta pronto
WILLTALK_TRIAGE_READY_WAIT_MS	12000	Espera maxima pelo cliente QR ficar pronto
WILLTALK_INTERNAL_HOST / _BASE_URL	127.0.0.1	Destino do loopback (util em Docker)
WILLTALK_ALLOW_LOCAL_UPSERT_NO_AUTH	-	Dispensa o Bearer em chamadas de 127.0.0.1 — apenas para desenvolvimento
VISION_API_BASE_URL / VISION_API_KEY / WILLTALK_VISION_MODEL / AI_API_KEY / XAI_API_KEY / OPENAI_API_KEY	-	Provedor de visao computacional

15.1.4 Webhooks

Variavel	Padrao	Funcao
WILLTALK_WEBHOOK_URL	-	Destino dos eventos de saida; vazio desliga o envio
WILLTALK_WEBHOOK_TOKEN	-	Bearer compartilhado — usado na saida e na validacao da entrada
WILLTALK_WEBHOOK_EVENTS	os quatro eventos	Lista CSV para filtrar o que e enviado
WILLTALK_WEBHOOK_MAX_CHARS	12000	Teto do historico consolidado
WILLTALK_WEBHOOK_ATTEMPTS	3	Tentativas por evento
WILLTALK_WEBHOOK_TIMEOUT_MS	8000	Timeout por tentativa
WILLTALK_TICKET_UPSERT_EMIT_WEBHOOKS	false	Reabilita webhooks a partir do ticket-upsert (risco de loop)
WILLTALK_AUTO_REPLY_ENABLED	false	Habilita a rota de resposta assistida do Cerebro
WILLTALK_AUTO_REPLY_ROUTE	/webhooks/cerebro/reply	Caminho aceito
WILLTALK_AUTO_REPLY_SOURCE	cerebro-operacional	Valor exigido no campo <i>origem</i>

15.2 Procedimento de subida

```
# 1. Dependencias
npm install
cd frontend && npm install && cd ..

# 2. Banco (uma vez): executar no Query Tool do pgAdmin, base "willtalk"
#   scripts/supabase-schema.sql

# 3. Seed de organizacao + admin
npm run seed-postgres

# 4. Backend (Next + Socket.IO + WhatsApp) -> http://localhost:4002
npm run dev
```



```
# 5. Interface do atendente (outro terminal) -> http://localhost:4001
cd frontend && npm run dev
```

```
# 6. Workers (opcional; exige REDIS_URL)
npm run worker
```

```
# Utilitário: gerar um JWT_SECRET forte
npm run jwt-secret
```

Credenciais iniciais do seed: admin@willtalk.local / admin123 — trocar imediatamente após o primeiro acesso.

15.3 Conectar o WhatsApp

1. Garantir WHATSAPP_PROVIDER=unofficial no .env.
2. Entrar na SPA com um usuário admin ou gestor e abrir **Painel** (/painel).
3. Clicar em **Gerar QR** e escanear com o WhatsApp do aparelho.
4. Aguardar o status mudar para **ready** — a partir daí as mensagens entram automaticamente.

15.4 Scripts de apoio

Script	Uso
iniciar-willtalk.bat	Sobe o ambiente completo
verificar-servicos.bat	Confere se as portas e serviços estão no ar
diagnostico-willtalk.sh	Diagnóstico geral do ambiente
test-n8n-ticket-upsert.bat	Teste interativo: pergunta mensagem, telefone e nome; permite testar direto no ticket-upsert local ou via webhook do n8n
teste-fluxo-completo-ia.bat	Exercita o caminho completo de triagem por IA
teste-mensagem-whatsapp.bat	Envio de mensagem de teste
scripts/verify-n8n-webhook.ps1	Valida se o webhook do n8n responde 200
scripts/test-upsert-send.ps1 / test-upsert-check.ps1	Envio e verificação do ticket-upsert em PowerShell
scripts/jwt-secret.mjs	Gera segredo JWT

Comportamento que confunde em teste: o mesmo telefone sempre reutiliza a mesma conversa. Se a triagem daquela conversa já foi concluída, novas chamadas ao ticket-upsert retornam shouldReply: false e nenhum menu aparece. Para repetir o teste do zero, use outro número ou reabra a triagem no banco/painel.

16. Avaliação técnica e recomendações

Esta seção registra a leitura de engenharia sobre o estado do código: o que está sólido, o que é dívida técnica consciente e o que representa risco real. É o material para priorizar o próximo ciclo.

16.1 Pontos fortes

- **Tratamento de falha de verdade.** Praticamente todo caminho externo (Cloudinary, Cerebro, webhook, Twilio, WhatsApp, Redis) degrada em vez de derrubar o fluxo principal. Mensagem do atendente é persistida mesmo se o envio falhar; encerramento de chamado não depende do Redis.
- **Guardas anti-loop pensados na prática.** As janelas de deduplicação, o cooldown de endpoint e a supressão de webhook no ticket-upsert claramente nasceram de incidentes reais e resolvem o problema certo.
- **Ponto único de ingestão.** Ter todos os canais convergindo para o ticket-upsert deixa a auditoria e o diagnóstico concentrados em um lugar só.
- **Validação consistente.** Zod em toda rota de escrita, com mensagens de erro estáveis.
- **Cuidado com o fuso horário.** Horário comercial e saudação usam Intl com America/Sao_Paulo, e não o relógio do servidor.
- **Trilha de auditoria.** Ações sensíveis gravam em audit_logs com metadados em JSONB.

16.2 Dívida técnica

Item	Impacto	Recomendação
Dois repositórios de dados paralelos — firestore-repo (50 KB) e supabase-repo (48 KB) mantidos manualmente em sincronia pelo seletor pick()	Alto	Escolher um backend. Se o Firestore não voltara a ser usado, remover a implementação: são ~50 KB de código que precisa ser alterado em duplicidade a cada mudança de modelo
Prisma declarado mas não usado — o schema está desatualizado em relação ao SQL real (faltam satisfaction_score, avatar_url, blocked, internal_note, contact_phone, tipo audio, quick_replies)	Médio	Ou adotar o Prisma de fato como fonte da verdade e migrar o acesso, ou remover a dependência e assumir supabase-schema.sql como schema autoritativo
Lógica de triagem triplicada — whatsapp-client, webhook Twilio e ticket-upsert implementam variações do mesmo roteiro	Alto	Extrair um único módulo de triagem e fazer os três canais chamarem a mesma função; hoje uma correção de regra precisa ser replicada em três arquivos
Dois interfaces mantidas — Next legado e SPA Vite	Médio	Aposentar o dashboard Next ou reduzi-lo a um redirecionamento para a SPA
Telas com dados falsos — Métricas e Cofre de Acesso leem mockData	Médio	A API de métricas já existe e já calcula satisfação; e só cabear. O Cofre precisa de modelo e API
SLA sem fechamento — prazo gravado, nenhum job de vencimento, evento nunca emitido	Alto	Chamar enqueueSLACheck na classificação e implementar o worker para emitir sla.breached. A interface já está preparada para receber
Workers vazios — sla-events e webhook-events apenas logam	Médio	Implementar ou remover, para não sugerir uma capacidade que não existe

16.3 Riscos de segurança a tratar

Risco	Severidade	Correção sugerida
Broadcast do Socket.IO sem sala por organização — todo evento vai para todos os sockets	Alta	Fazer o socket entrar em uma sala por organizationId no handshake e trocar io.emit por io.to(sala).emit
CORS do Socket.IO em origin: ""	Media	Restringir a lista de origens conhecidas
GEMINI_API_KEY injetada no bundle do Vite	Alta	Mover as chamadas ao Gemini para uma rota do backend
WILLTALK_ALLOW_LOCAL_UPSERT_NO_AUTH dispensa o Bearer em chamadas locais	Media	Garantir que fique desligado fora de desenvolvimento
JWT_SECRET tem valor padrão embutido no código	Media	Falhar a inicialização se a variável não estiver definida em produção
generateSignedUrl ignora o parâmetro de expiração	Media	Passar expires_at ao Cloudinary; hoje a URL assinada não expira como esperado
Limpeza do Cloudinary usa apenas resource_type: image	Baixa	Audio e documentos ficam orfãos; iterar também sobre video e raw
Isolamento multi-tenant depende exclusivamente do filtro na rota	Media	Considerar RLS no PostgreSQL como segunda barreira

16.4 Escalabilidade

- O cliente WhatsApp em memória global impede rodar mais de uma instância do servidor.
- Os guardas anti-loop em Map teriam que migrar para o Redis em um cenário com vários nós.
- listConversations carrega a lista inteira e filtra em memória em vários pontos (resolução de ticket_ref, snapshot de webhook, alvo da resposta do Cerebro). Funciona bem no volume atual, mas vira gargalo com milhares de conversas — vale trocar por consultas indexadas diretas.
- Não há paginação no inbox nem no histórico de mensagens.

16.5 Governança do repositório

O repositório tem apenas **dois commits** e o estado atual traz **19 arquivos modificados e cerca de 10 não rastreados sem commit**, incluindo módulos críticos: lib/whatsapp-client.ts, lib/supabase-repo.ts, toda a pasta app/api/webhooks/cerebro/ e app/api/webhooks/n8n/, e a pasta docs/. Todo o trabalho de integração com o Cerebro existe apenas em disco. **Este é o risco mais imediato do projeto:** uma falha de máquina hoje custaria o trabalho recente inteiro. Recomendação: commitar em unidades lógicas e publicar em um remoto antes de qualquer outra tarefa.

16.6 Roteiro sugerido

Prioridade	Ação
1 — imediata	Commitar e publicar o trabalho pendente em um repositório remoto
2 — segurança	Salas por organização no Socket.IO; mover o Gemini para o backend; exigir JWT_SECRET em produção
3 — funcionalidade	Fechar o ciclo de SLA (enfileirar, detectar vencimento, emitir o evento)
4 — funcionalidade	Cabeçar as telas de Métricas e Cofre de Acesso na API real
5 — manutenibilidade	Unificar a triagem em um único módulo compartilhado pelos três canais
6 — manutenibilidade	Decidir o backend de dados e remover a implementação que não será usada

Prioridade	Acao
7 — qualidade	Cobrir com testes ao menos a triagem, a deduplicacao e a normalizacao de telefone — hoje as regras mais criticas nao tem rede de protecao

17. Glossario

Termo	Significado no contexto do WillTalk
Cerebro Operacional	Servico externo (projeto Mavo AI / chat-inteligente) que hospeda o orquestrador de IA e o RAG; decide o que responder ao cliente
Demanda / Fila (Queue)	Categoria de atendimento com opcao de menu, cor de card e SLA proprio
Triagem	Fase automatizada antes do atendente humano assumir a conversa
triage_completed	Marca o fim da fase de bot; a partir dai o humano conduz
menu_attempts	Contador com duplo uso: tentativas invalidas de menu e rodadas de investigacao da IA
ticket-upsert	Rota central de ingestao onde toda mensagem entra e toda decisao de resposta e tomada
Provider unofficial	Canal WhatsApp por whatsapp-web.js com QR Code, sem API oficial
Dry run	Modo em que as respostas sao gravadas no historico mas nao enviadas ao cliente real
Eco de historico	Situacao em que o texto consolidado enviado ao n8n retorna como mensagem de entrada
Loopback	Chamada HTTP que o servidor faz a si mesmo (127.0.0.1) para reaproveitar a rota de ingestao
Painel	Tela administrativa de conexao do WhatsApp por QR Code (/painel)
Cofre de Acesso (Vault)	Tela para guardar credenciais de acesso remoto a clientes; ainda com dados de exemplo

Documento gerado a partir da leitura integral do codigo-fonte em C:\willydev\willtalk. Reflete o estado do sistema em 22 de julho de 2026, incluindo alteracoes ainda nao commitadas.